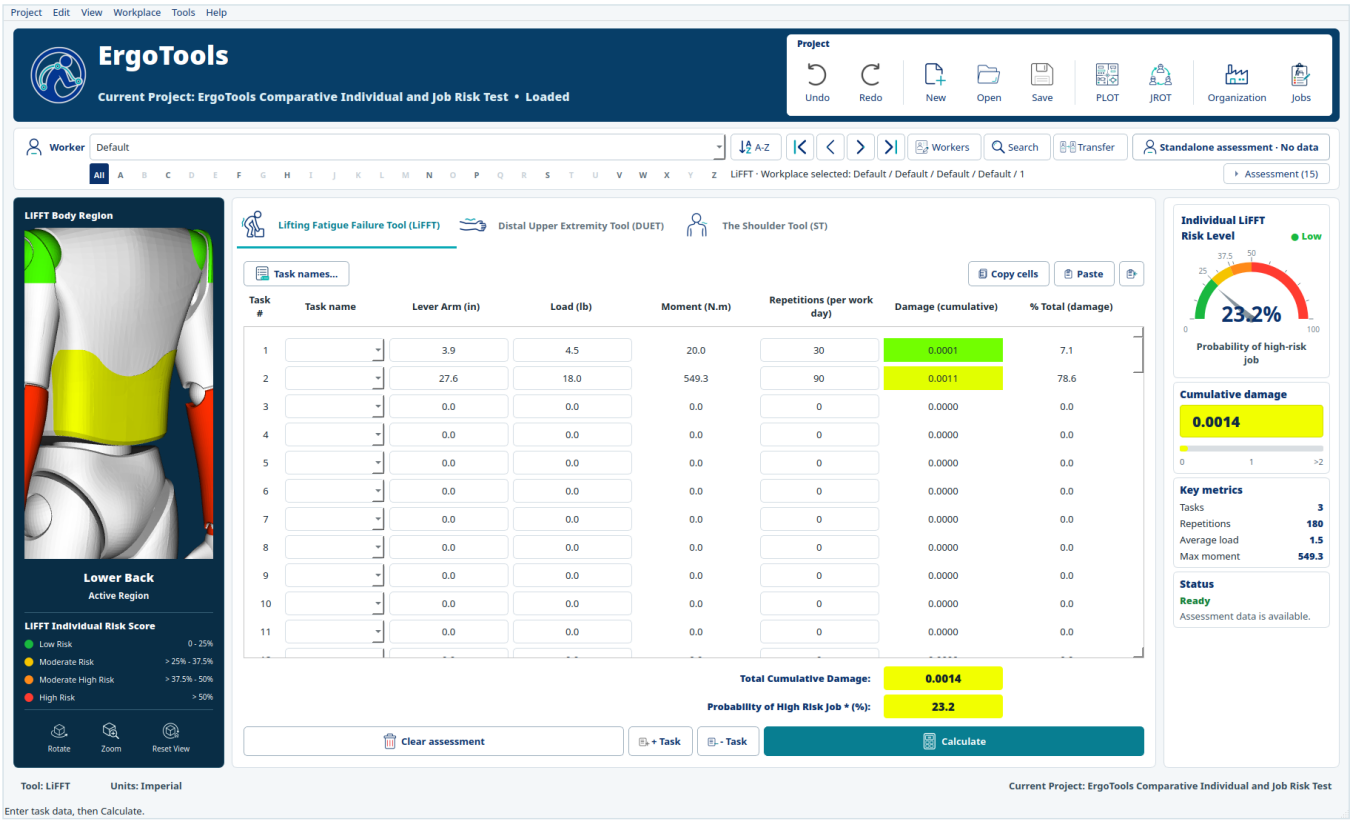
ErgoTools

SOFTWARE INSTALLATION AND GITHUB GUIDE

Install. Launch. Update. Get help.

Windows | macOS | Linux

A practical guide to the source checkout and desktop application



ErgoTools combines task assessment, anatomical visualization and risk summaries.

CLONE ONCE

Keep one stable code folder.

LAUNCH EASILY

Use a shortcut or alias.

UPDATE SAFELY

Keep your projects separate.

REPORT CLEARLY

Send steps, images and logs.

For users installing or maintaining a local copy; no programming experience is assumed.

START HERE

Find the instructions you need

Install only for your operating system. Return to the shared chapters when you need to update or report a problem.

Before you install **3-4**

Windows installation and desktop shortcut **5-12**

macOS installation and desktop alias **13-20**

Linux installation and launcher **21-23**

Your first personal project **24**

Updating code and libraries **25-27**

Version identification and recovery **28-29**

Reporting a bug and taking screenshots **30-33**

Finding logs and capturing console output **34-37**

Attaching evidence and following up **38-39**

Quick reference **40**

References and image credits **41**

A useful first read
Read pages 3-4, follow your OS chapter, then use page 24 to create or save your own project copy.

Commands are selectable text. The command companion linked from the repository is useful if your PDF viewer changes spacing when copying.

What you need

Create a free GitHub account and verify its email before using GitHub Desktop or posting a bug report.

Create an account at GitHub and download GitHub Desktop. Sign in with your own account; screenshot usernames are examples.

Platform	Prepare
Windows	Windows 10/11 x64, PowerShell and GitHub Desktop.
macOS	macOS 12+ for GitHub Desktop. Choose Intel or Apple silicon downloads to match About This Mac.
Linux	An x86-64 graphical desktop, Git, a terminal and working OpenGL/Qt system libraries.

Three tools, three different jobs

Tool	What it does
GitHub Desktop / Git	Copies and updates the source code from the online repository.
uv + Python 3.13.14	Creates the local Python environment and installs the pinned libraries.
ErgoTools launcher	Starts the application from that existing environment. It does not update code.

Keep these locations handy
Code folder: _____
Personal project folder: _____
Shortcut or alias name: _____

Internet is needed to download/install/update code and libraries. Ordinary local assessment and optimization work can run offline after setup.

Folders and commands without guesswork

Keep one stable code folder and store your own .ergprj project files outside it.

Item	Example / meaning
Repository on GitHub	The online ErgoTools source. Read its README; do not create a new repository.
Local checkout	ErgoTools-code/ErgoTools inside your home folder. This contains tools, src, pyproject.toml and uv.lock.
.venv	The Python environment created inside the checkout. Do not copy it between computers or move it separately.
Personal projects	A separate folder such as Documents/ErgoTools projects. Save your own files here.

Understand the path before pasting

$HOME is your current user's home folder in PowerShell, Bash and zsh. It fills in your username automatically. ~ also means your home folder in Finder's Go to Folder. Keep quotes around paths that contain spaces.

```
cd "$HOME\ErgoTools-code\ErgoTools"
cd "$HOME/ErgoTools-code/ErgoTools"
```

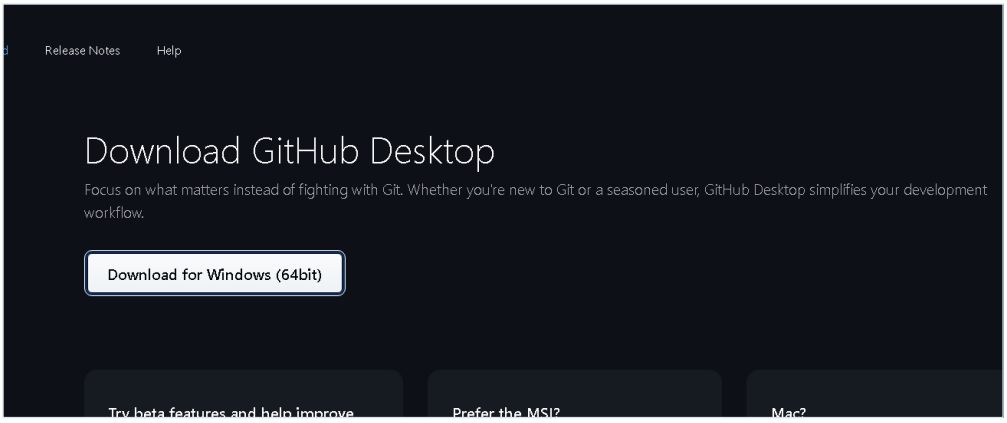
The first line is Windows; the second is macOS/Linux. Use only your OS line. If you chose a different clone location, replace the entire quoted path with the folder you chose. In Desktop, use Show in Explorer/Finder; then copy that folder's address.

Copy only the command
Do not copy PS>, $, a username, the terminal prompt or screenshot text. PowerShell uses a final backtick (`) to continue a command; macOS/Linux use a final backslash (\). Paste all lines of one block together.

You do not need to activate .venv or change your computer's system Python. Run the supplied launcher from the same checkout you installed.

Install GitHub Desktop and clone

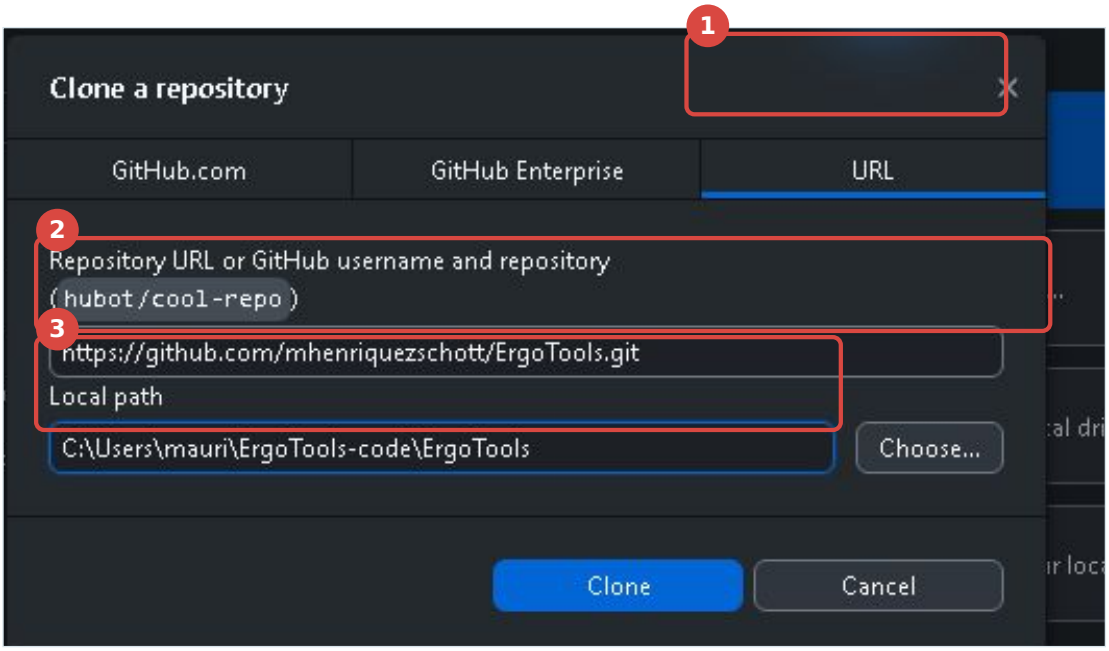
Download the 64-bit Windows installer, run it, and sign in to your GitHub account. Use a normal user account.



The official download page offers the Windows installer.

In Desktop choose **File > Clone Repository**, then select the **URL** tab. The GitHub.com tab searches account repositories; it is not the place to paste the full clone URL.

```
https://github.com/mhenriquezschott/ErgoTools.git
```

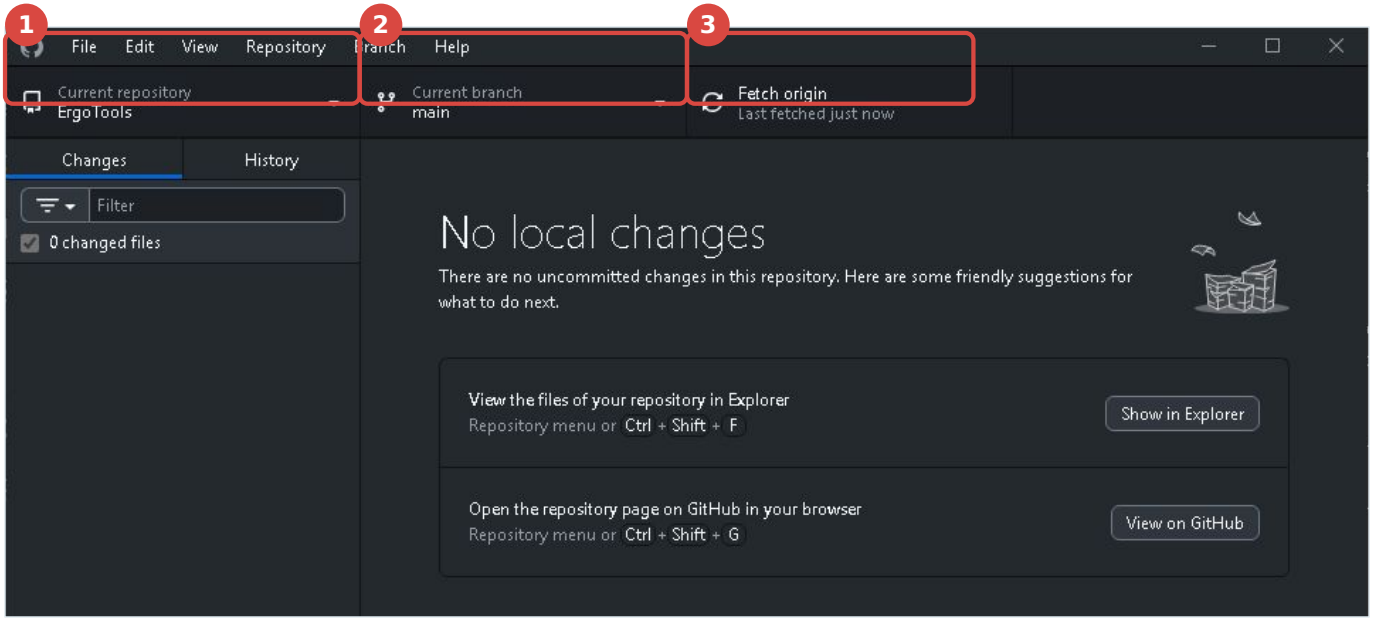


1 Select URL. 2 Paste the repository address. 3 Choose your local checkout folder.

Recommended folder: C:\Users\your-name\ErgoTools-code\ErgoTools. Choose a local folder you own. Avoid a temporary download directory.

Confirm the checkout and open PowerShell

Wait for cloning to finish. Select ErgoTools as the current repository and main as the current branch.



The clean repository view is the starting point for installation.

The Changes view should show no changed files. A clean clone is ready; you do not need to commit or push anything.

Open Windows Start, type **PowerShell**, then open it normally. Administrator mode is not needed. A terminal window accepts the commands on the next pages.

```
cd "$HOME\ErgoTools-code\ErgoTools"
```

If you used another location
In Desktop click Show in Explorer, or use Repository > Show in Explorer. Copy the checkout folder's address from Explorer's address bar. Use cd "copied folder path" in PowerShell.

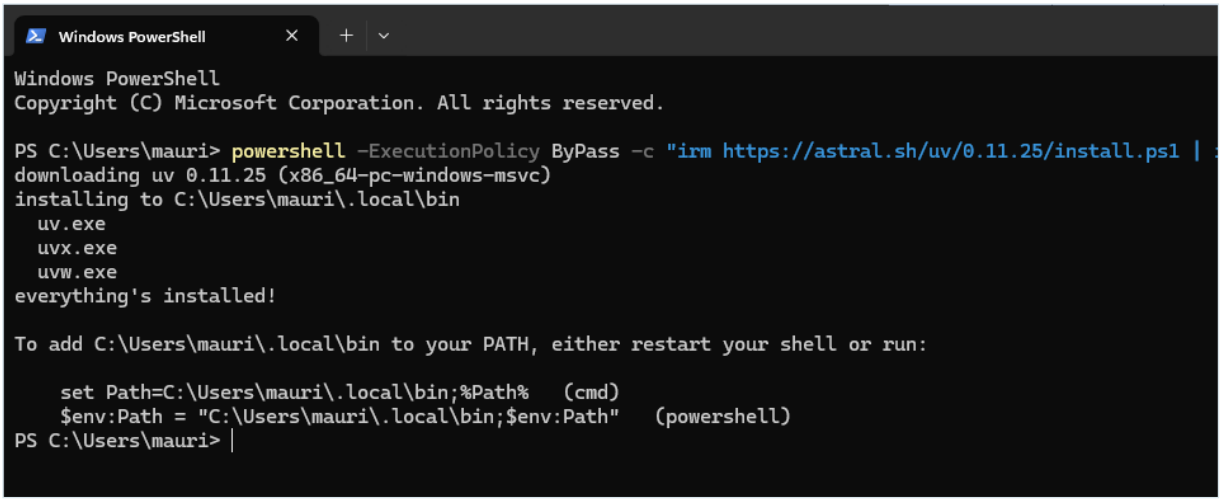
The terminal should now be in the folder containing pyproject.toml and tools. The prompt changes to that directory; do not type the prompt itself.

Install uv and reopen PowerShell

uv manages the Python version and the local environment. You install it once per user.

```
powershell -ExecutionPolicy ByPass `
  -c "irm https://astral.sh/uv/0.11.25/install.ps1 | iex"
```

Copy both lines together, including the backtick at the end of line 1. Wait until installation completes and the PS prompt returns.



Installation ends with everything installed and a PATH reminder. Reopen PowerShell before continuing.

Restart the shell, not the installation

Close PowerShell completely, reopen it from Start, and run:

```
uv --version
```

Check before continuing
The output should identify uv 0.11.25. If uv is not recognized, verify that the installer finished and open a new PowerShell window. A tab in an old terminal can keep the old PATH.

Then return to your checkout with the cd command from page 6. Do not keep installing Python packages into the Windows system Python.

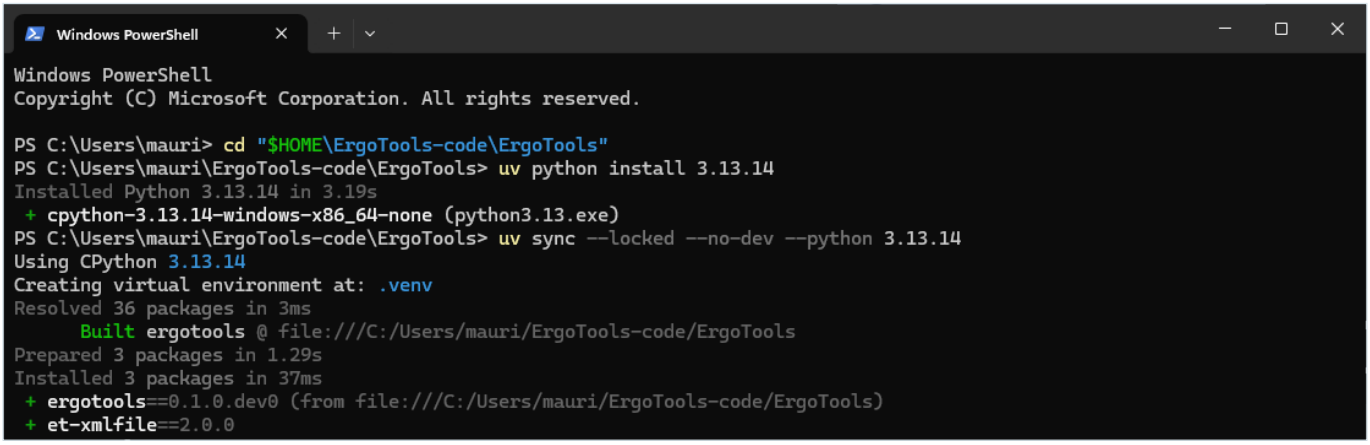
Install Python and create the environment

Run these commands from the ErgoTools checkout, one block at a time.

```
cd "$HOME\ErgoTools-code\ErgoTools"
uv python install 3.13.14
```

Wait for Python 3.13.14 to finish installing. If that exact version is already installed, uv can reuse it.

```
uv sync --locked --no-dev --python 3.13.14
```



Look for Python 3.13.14, creation of .venv, and successful installation of ErgoTools.

What sync does
This creates the editable application environment. On Windows, the graphical desktop libraries are installed separately on the next page. This is a required second step.

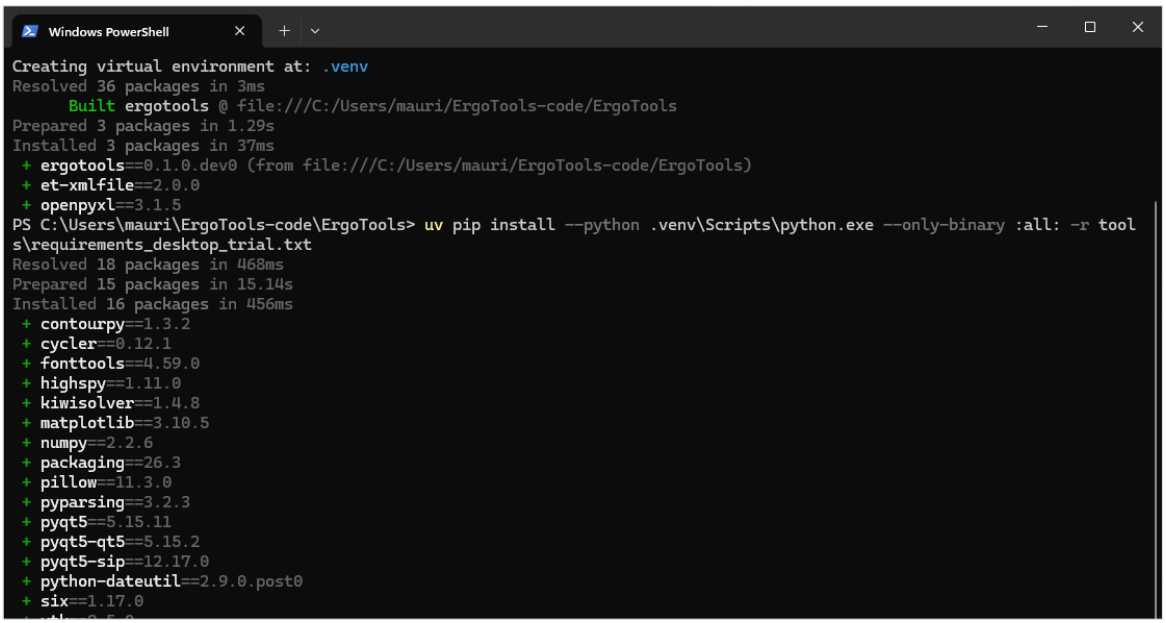
If the command says pyproject.toml cannot be found, you are in the wrong folder. Use Show in Explorer and return to the checkout before retrying.

Install the desktop libraries and launch

Use the repository's pinned requirements. Do not substitute versions from an older branch or another environment.

```
uv pip install --only-binary :all: `
  --python .venv\Scripts\python.exe `
  -r tools\requirements_desktop_trial.txt
```

Paste all three lines together. The two final backticks continue the command. Wait for the PS prompt to return after the package list.



The installed libraries include PyQt5, Qt, VTK and HiGHS. Package counts and timing can vary.

```
.\tools\run_windows.cmd
```

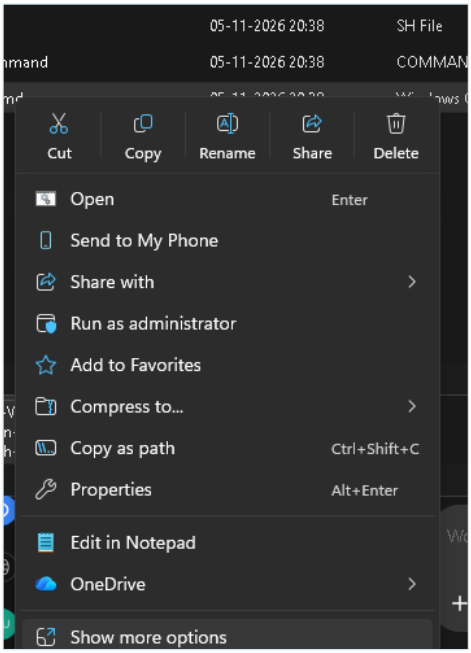
What you should see

The ErgoTools main window and 3D model open. Without a project, the task area is empty and project-related controls can be disabled. That is expected; installation does not create worker data.

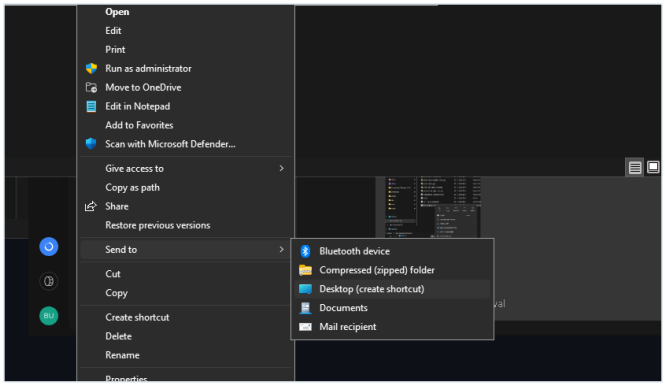
Windows uses Qt 5.15.2 because Qt 5.15.17 has no Windows wheel. Keep the other pinned library versions unchanged.

Create a desktop shortcut

In Desktop use Show in Explorer, open tools, and find run_windows.cmd. Keep that original file inside tools.

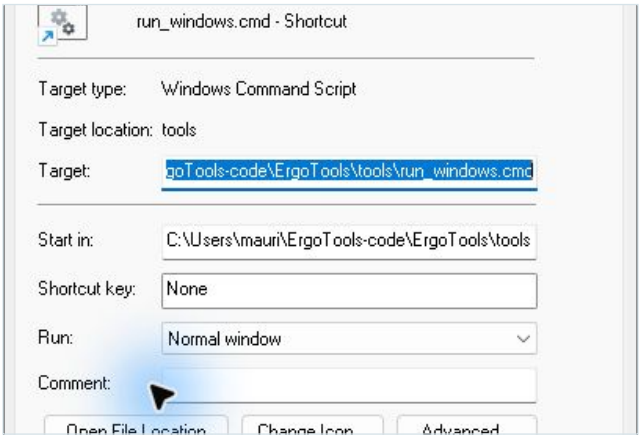


1 On Windows 11, right-click and choose Show more options.



2 Choose Send to > Desktop (create shortcut).

On Windows 10, Send to may appear directly in the first context menu. Rename the new desktop shortcut **ErgoTools**. Close the app, then double-click the shortcut to check it.



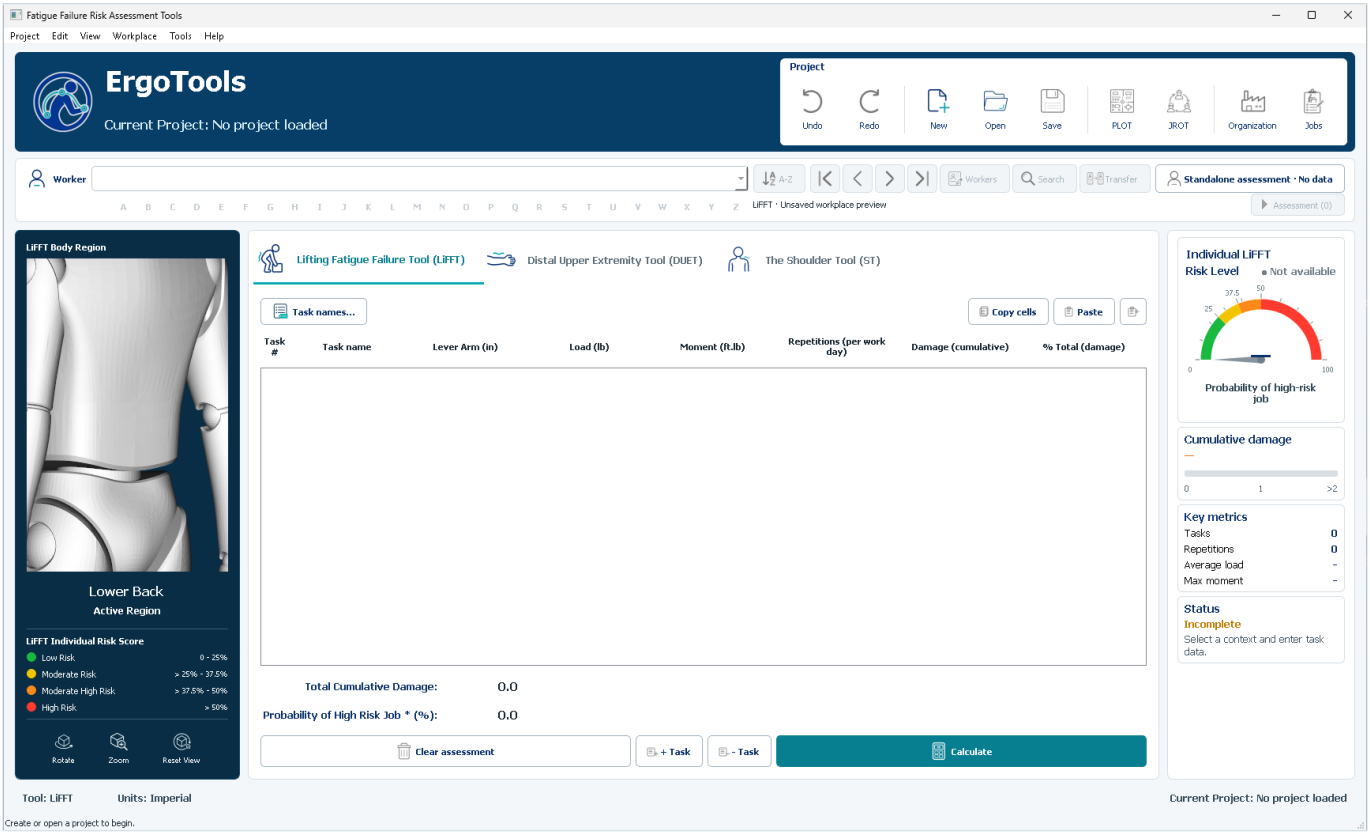
Shortcut properties: verify the target points into your checkout's tools folder.

Check the target
Right-click the shortcut > Properties > Shortcut. Target must end with **ErgoTools\tools\run_windows.cmd**. If it points to an older folder, you will launch older code.

Shortcut, not a copied script
The launcher finds .venv relative to its own location. Moving just run_windows.cmd onto the Desktop breaks that relationship. The shortcut can move; the original launcher stays in tools.

Confirm the main window opens

Launch once from the desktop shortcut. The terminal may remain open behind the application.



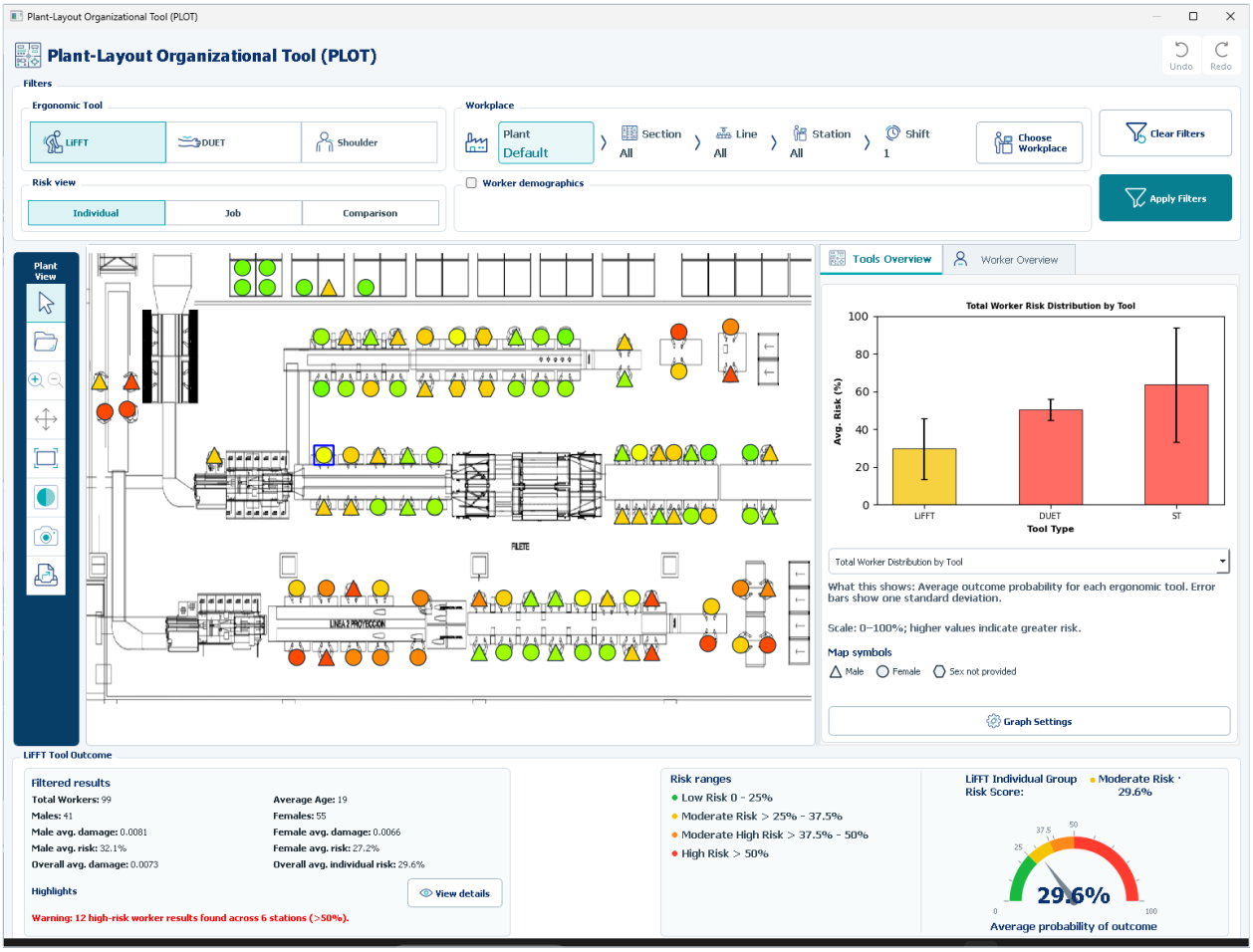
Native Windows capture: the application opens with the anatomical model visible.

Check	Expected
3D anatomy	The model is visible. Use Rotate/Zoom/Reset View to check interaction.
No project loaded	Empty task inputs and disabled project controls are normal until a project is opened.
A personal test copy	Use Project > Open, or the Open toolbar button. Start with a copy of a bundled sample.

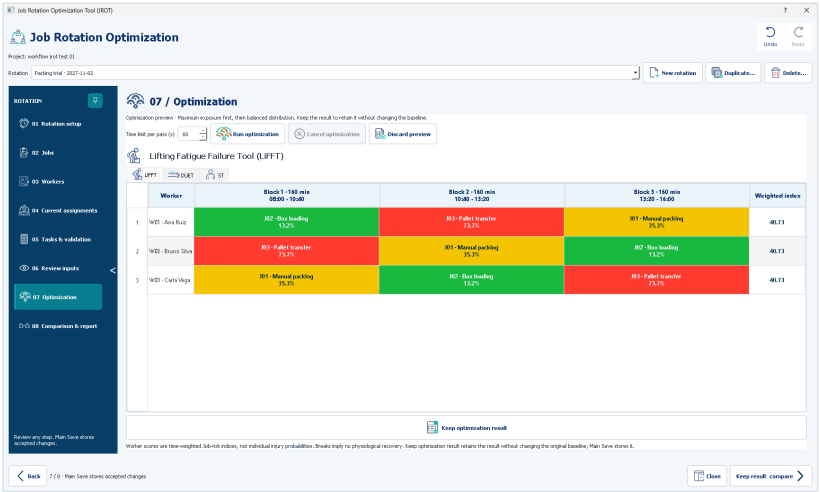
Keep a working launch link
Your shortcut points to the same checkout after a normal code update. It does not fetch updates itself. Save your project and close the app before pulling new code.

Open PLOT and view a rotation result

These Windows captures show the application with example project data. Continue with your own project copy, not the bundled original.



PLOT on Windows: plant map, worker symbols, population chart and outcome summary are visible.



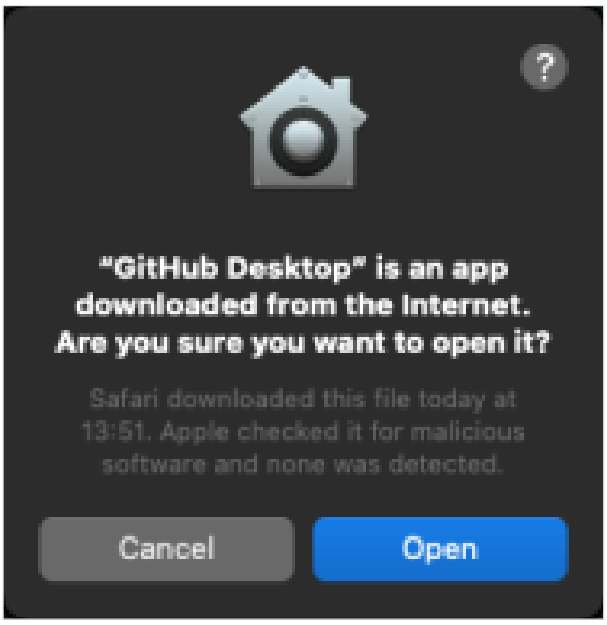
JROT on Windows: an optimization result is displayed for three tools. Viewing a stored result and running a fresh optimization are different actions.

For a fresh check, choose Run optimization, wait for the result, use Keep optimization result, then Main Save if you want to retain it. Shared project guidance starts on page 24.

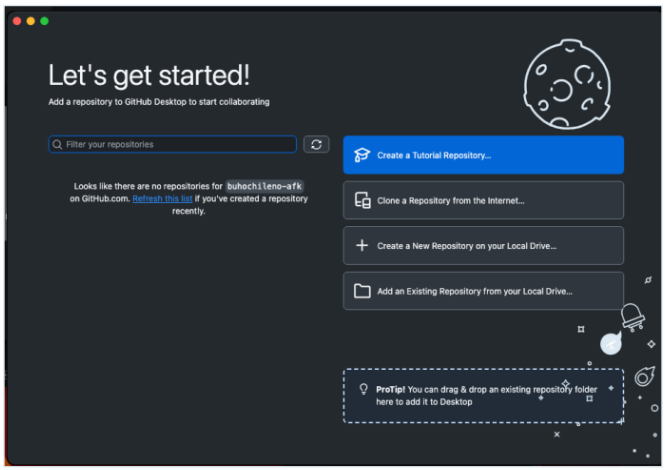
Install GitHub Desktop

Download GitHub Desktop for your Mac: choose Intel or Apple silicon to match Apple menu > About This Mac.

Open the download and move GitHub Desktop into Applications. Launch it from Applications. Confirm Open if macOS asks about an application downloaded from the internet, then sign in to your GitHub account.



The first-open confirmation identifies GitHub Desktop.



Continue through the welcome screen and sign in.

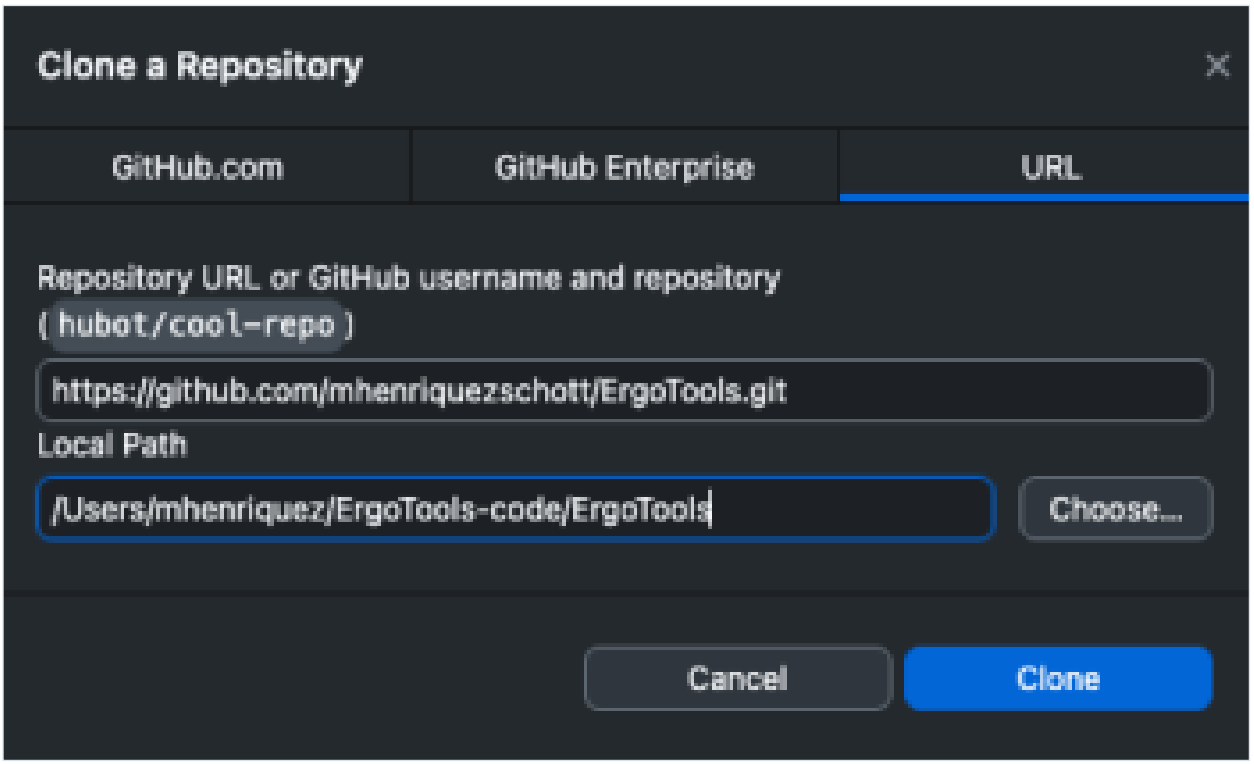
Use the mouse if shortcuts differ
Menus and contextual menus provide every essential action in this guide. On a virtual machine, keyboard shortcuts may be intercepted by the host computer.

GitHub Desktop manages the source checkout. It is separate from the ErgoTools application you will launch later.

Clone the repository using its URL

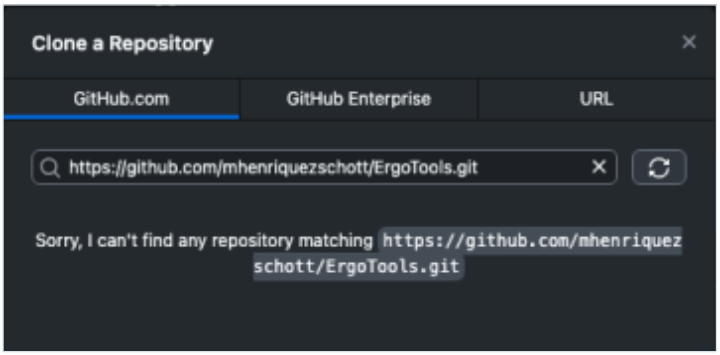
In GitHub Desktop choose File > Clone Repository and switch to URL before pasting the address.

```
https://github.com/mhenriquezschott/ErgoTools.git
```



Use URL, enter the repository address, and choose the local folder before Clone.

Use a stable folder such as **ErgoTools-code/ErgoTools** inside your home folder. After cloning, confirm that Current repository is ErgoTools and Current branch is main.



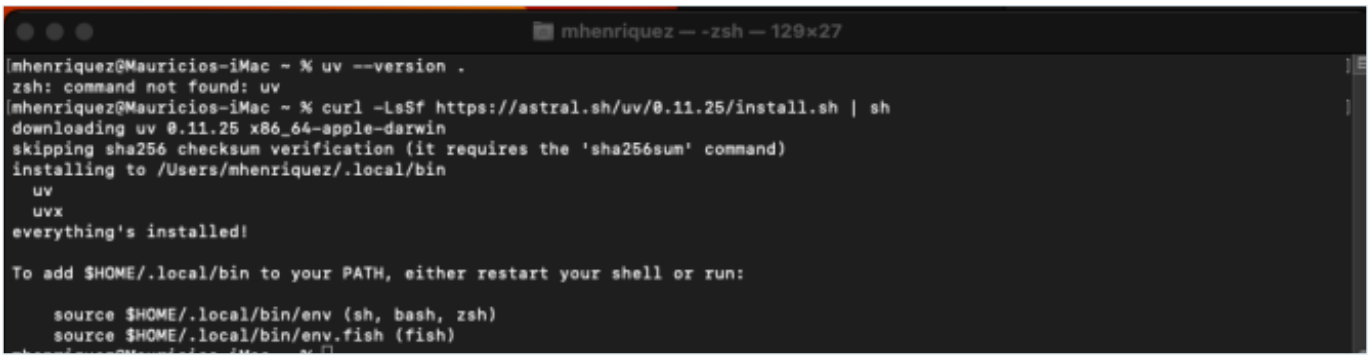
If the address is rejected on GitHub.com, switch to the URL tab instead of changing the address.

Write down the actual folder
Use Show in Finder in the repository view. If you chose a different location, use that location in the terminal commands and when creating your alias.

Install uv and open a new Terminal

Open Applications > Utilities > Terminal. uv installs the required Python and libraries into the checkout.

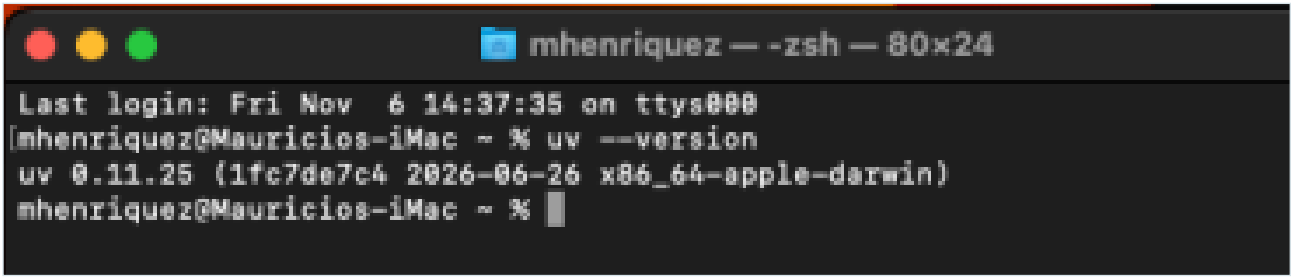
```
curl -LsSf https://astral.sh/uv/0.11.25/install.sh | sh
```



The installer reports where uv was placed and how to refresh the shell.

Close this Terminal window and open a new one so the new command is available. Check it, then enter your checkout folder.

```
uv --version
cd "$HOME/ErgoTools-code/ErgoTools"
```



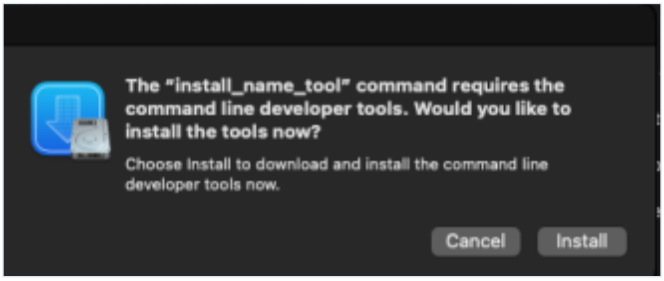
A version number confirms that Terminal can find uv.

If uv is not found
Open a new Terminal, not just a new command line in the existing window. If needed, follow the PATH instruction printed by the installer. Do not substitute a system Python for the pinned environment.

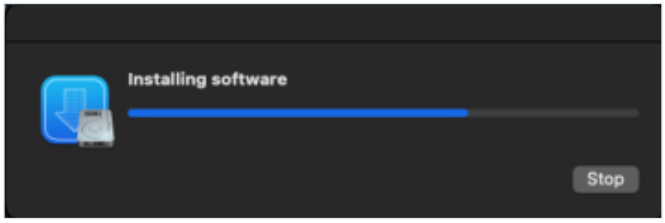
Install Python and Apple command-line tools

Python installation may trigger an Apple developer-tools prompt. Complete it before continuing.

```
uv python install 3.13.14
```



Choose Install when macOS requests command-line developer tools.



Wait for installation to finish; the full Xcode application is not required.

If the tools were not offered and a command reports that they are missing, request them and finish the installation dialog:

```
xcode-select --install
```

If Python was downloaded before the tools were available and uv reports an install_name_tool or patching error, reinstall that Python after the tools finish:

```
uv python install 3.13.14 --reinstall
```

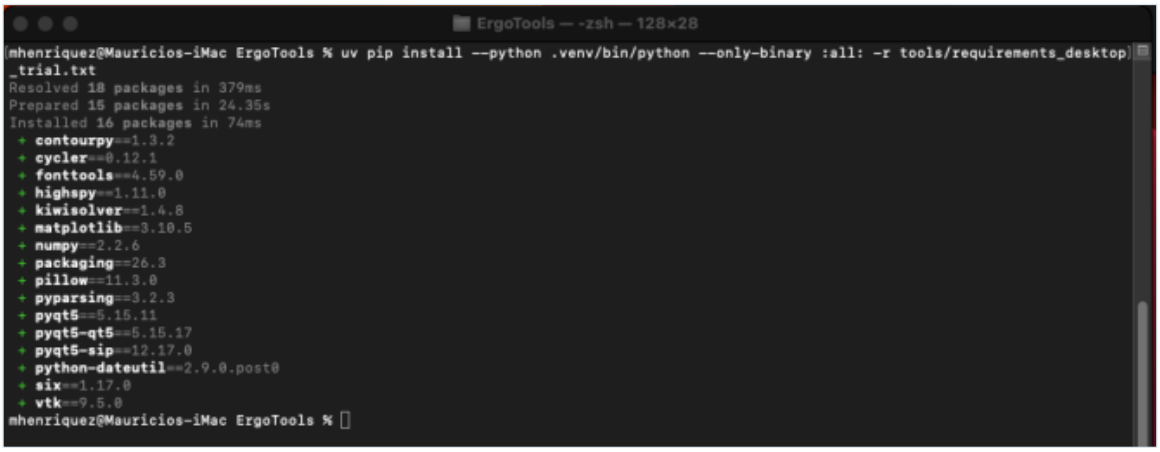
An ordinary rerun may do nothing
“Already installed” does not repeat a failed post-install patch. Use --reinstall for this specific recovery, then continue with the environment setup.

Create the environment and launch

Run these commands from the checkout folder. Wait for each to finish successfully before continuing.

```
cd "$HOME/ErgoTools-code/ErgoTools"
uv sync --locked --no-dev --python 3.13.14
```

```
uv pip install --only-binary :all: \
  --python .venv/bin/python \
  -r tools/requirements_desktop_trial.txt
```



The desktop-library installation completes inside .venv.

```
chmod +x tools/run_macos.command
./tools/run_macos.command
```

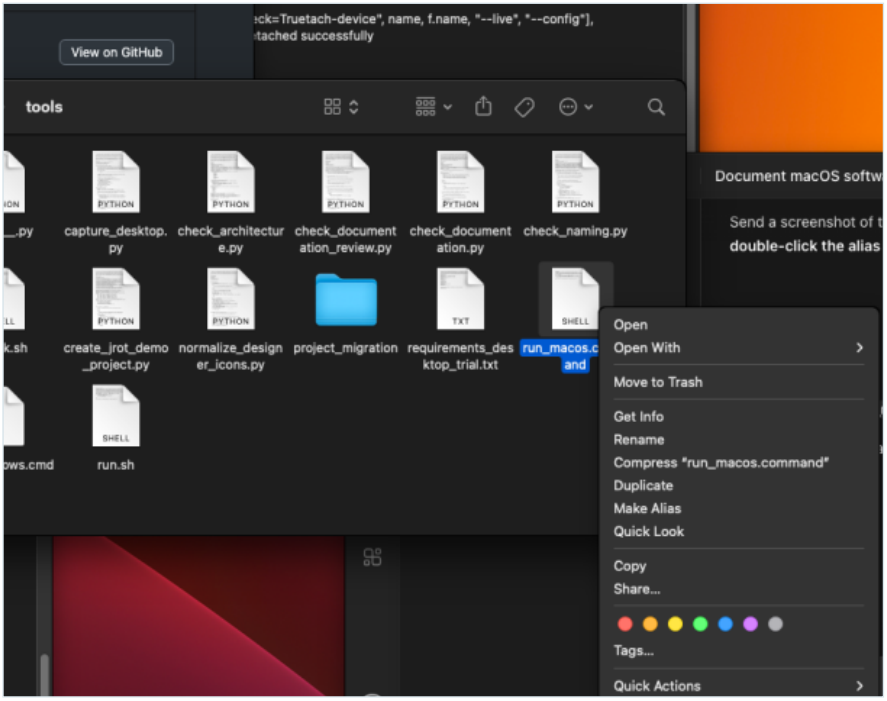
Two installation steps belong together
uv sync installs the application and core libraries. The requirements command installs the macOS desktop libraries. Repeat both after an update that refreshes the environment.

The .command launcher opens a terminal and starts ErgoTools. Keep that terminal available if you need to capture an error.

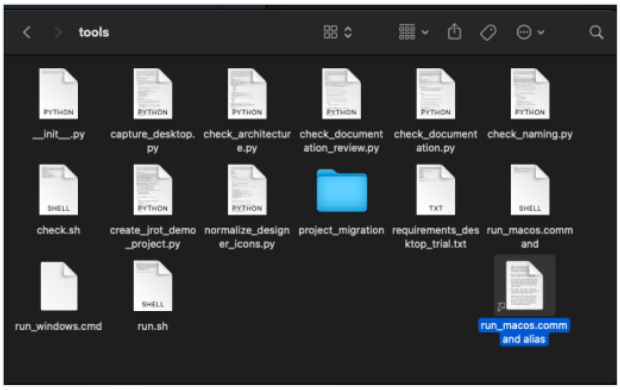
Make a desktop alias

Keep the original launcher inside tools. Put only an alias on the Desktop so it continues to work after code updates.

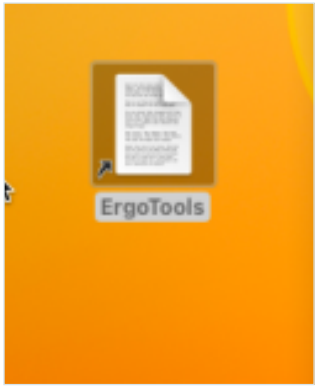
In GitHub Desktop choose **Show in Finder**, open the **tools** folder, then locate **run_macos.command**. Control-click or right-click it and choose **Make Alias**.



The contextual menu creates an alias to the launcher.



Move the new alias, not run_macos.command, to the Desktop.

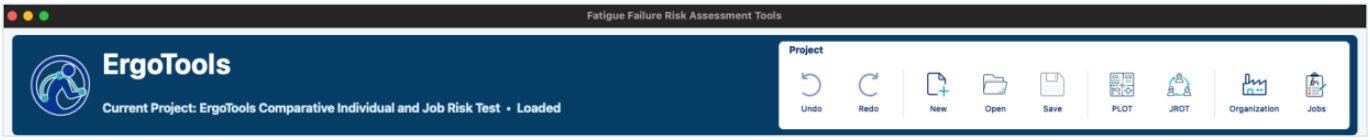


Rename the alias ErgoTools and double-click it to launch.

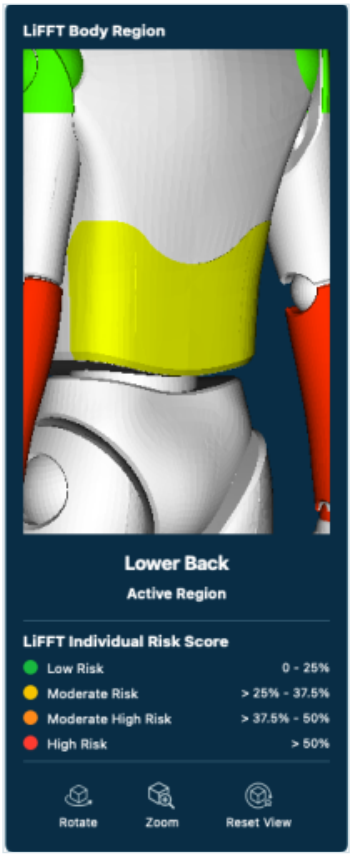
If the checkout moves
Recreate the environment in its new location and make a new alias. A normal Fetch/Pull in the same folder does not require a new alias.

Check the application and 3D view

Open the alias. With a project loaded, the main window combines task inputs, the anatomical model and a risk summary.



Native macOS application header and project controls.



The anatomical view should render; try Rotate and Reset View.



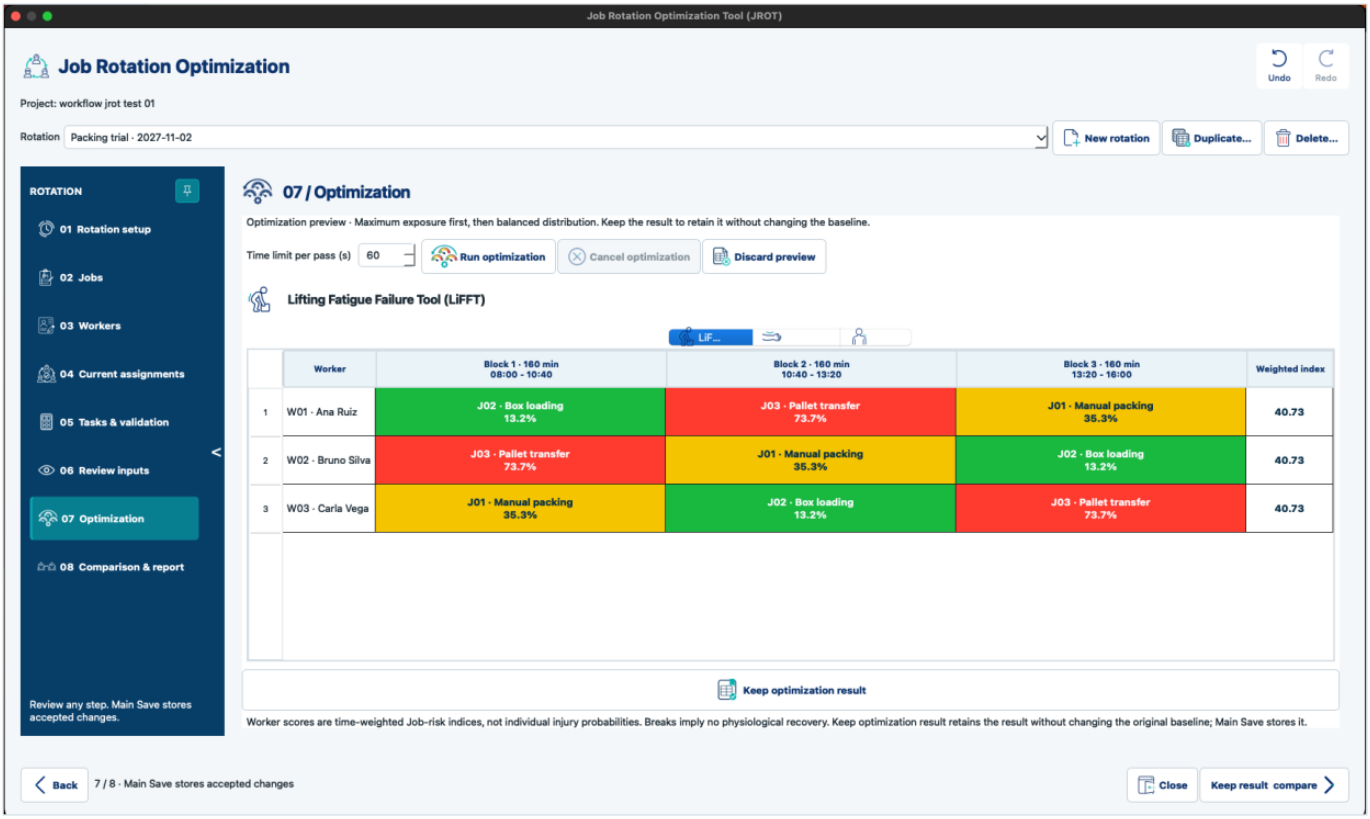
The gauge and cumulative-damage panel show the selected assessment result.

These detail views show the application running on macOS. An empty grid, unavailable gauge and disabled project actions are normal before you open a project and select an assessment.

A quick visual check
Check labels, button text and the bottom controls. If something is clipped, record the display resolution, scaling and system appearance with a screenshot and the launch log.

Check a rotation calculation

Open a personal copy of a sample project, then JROT. A fresh optimization checks the solver as well as the interface.



Native macOS optimization preview. Each column is a time block; each row is a worker.

1. Run a fresh optimization

Use Run optimization and wait for the calculation to finish. A previously displayed saved schedule alone does not exercise the solver.

2. Retain it deliberately

Keep optimization result retains the result without replacing the original baseline. Use the main Save control to write the project file.



Keep the preview only when you want it retained with this rotation.

Close and reopen your own project to check the saved data. Bundled samples are for copying, not for storing your work.

Clone, install and launch

Use a graphical Linux session with Git, curl and working OpenGL/Qt system libraries. This route uses the terminal rather than GitHub Desktop.

```
mkdir -p "$HOME/ErgoTools-code"
git clone --branch main --single-branch \
  https://github.com/mhenriquezschott/ErgoTools.git \
  "$HOME/ErgoTools-code/ErgoTools"
cd "$HOME/ErgoTools-code/ErgoTools"
```

```
curl -LsSf https://astral.sh/uv/0.11.25/install.sh | sh
```

Open a new terminal after installing uv. Return to the checkout, then run:

```
cd "$HOME/ErgoTools-code/ErgoTools"
uv python install 3.13.14
uv sync --locked --no-dev --extra desktop --python 3.13.14
bash tools/run.sh
```

If a system library is missing
Install the matching Qt/XCB/OpenGL runtime package for your distribution. Keep the pinned Python packages unchanged. Copy the exact missing-library error into a report if you need help.

The launcher configures the local environment and Qt platform handling. Use it instead of calling an unrelated system Python.

Add an application-menu launcher

This optional entry starts the checkout you installed. Run it from the repository root so $PWD expands to the correct folder.

```
mkdir -p "$HOME/.local/share/applications"
cat > "$HOME/.local/share/applications/ergotools.desktop" <<EOF
[Desktop Entry]
Type=Application
Name=ErgoTools
Exec=bash "$PWD/tools/run.sh"
Icon=$PWD/src/ergotools/resources/icons/ergotools_logo.png
Terminal=true
Categories=Science;Education;
EOF
```

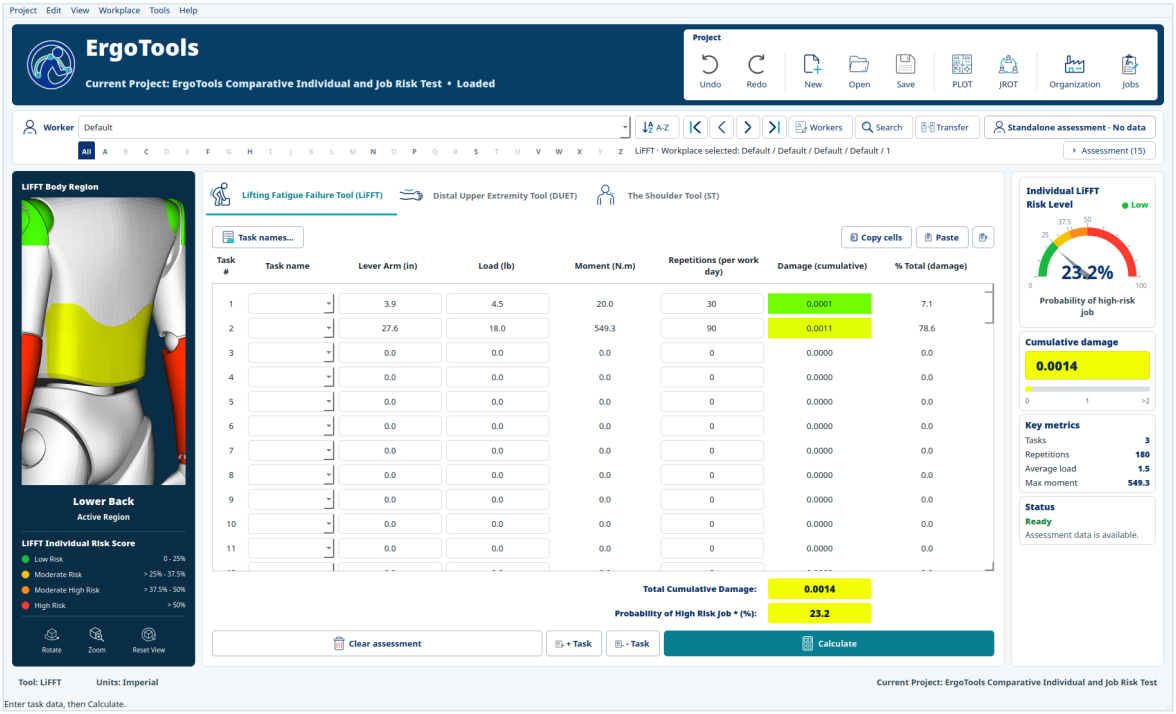
Find ErgoTools in your application menu and pin it using your desktop environment. Terminal=true keeps console output available behind the application.

Use the checkout location, not a project path
The entry points to tools/run.sh. Project files can live anywhere you normally keep your research data. Recreate the launcher if you move the checkout.

Some desktops require a copied .desktop file to be marked executable or trusted before launching it from the Desktop. Using the application menu avoids that extra desktop-file step.

Confirm rendering and the plant map

Open a project copy and inspect both the task-assessment view and PLOT.



Linux: task inputs, 3D anatomy and risk summary in the main application.



Linux: a populated comparative plant map with many workplace assessments.

Try a fresh single-tool and multi-tool optimization in JROT when checking a new machine. Save only your personal test copy.

Keep project data separate from code

The code checkout is updated from GitHub. Your .ergprj files are your own data and should live in a separate folder.

1. Create a personal folder

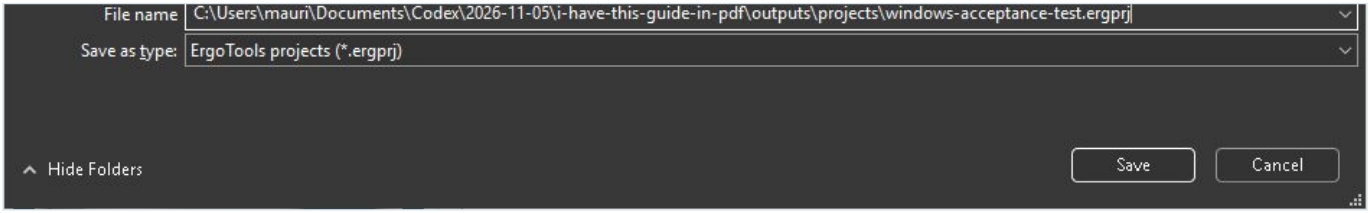
For example, create Documents/ErgoTools projects in Explorer or Finder. Choose a location included in your normal backups.

2. Start new or open a copy

Use Project > New for a new project, or copy a bundled file from test_projects/native into your personal folder before opening it.

3. Name and save the project

Choose your own filename and confirm the Project Details dialog if it appears after Save As. Check the project name in the main header.



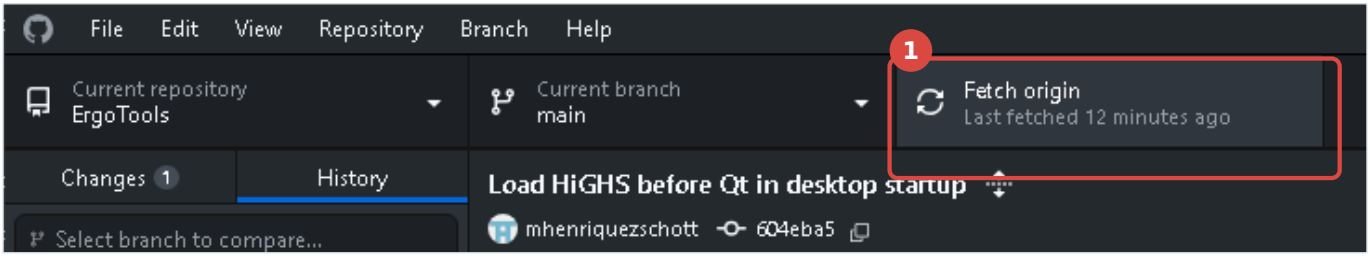
The file dialog chooses a destination and filename. Use your own project folder; names in captures are examples.

Code updates do not save your work
Use the main Save control before closing ErgoTools. Fetch/Pull updates program files; it is not a project backup. Keep a separate backup of important .ergprj files.

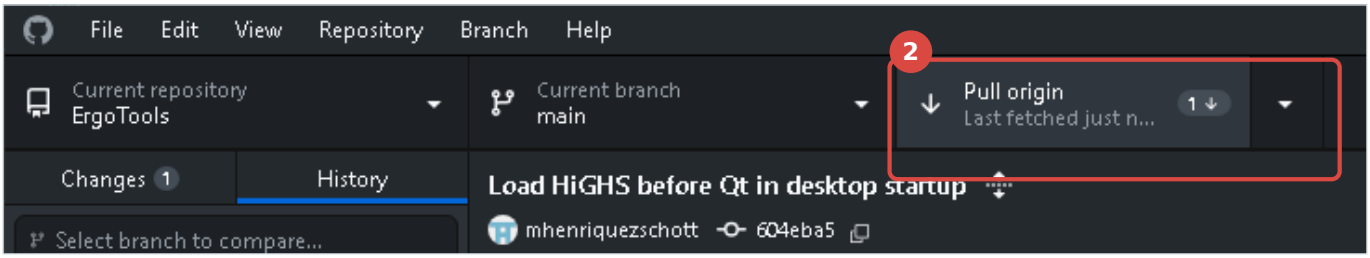
When checking a new installation, reopen your saved project and confirm a known input and result. Use the project menu to open its folder if you forget where it was saved.

Fetch and pull in GitHub Desktop

Save your project, close ErgoTools, then select the same repository and main branch in Desktop.



1 Fetch origin checks whether GitHub has new commits.



2 Pull origin downloads the available commits into this checkout.

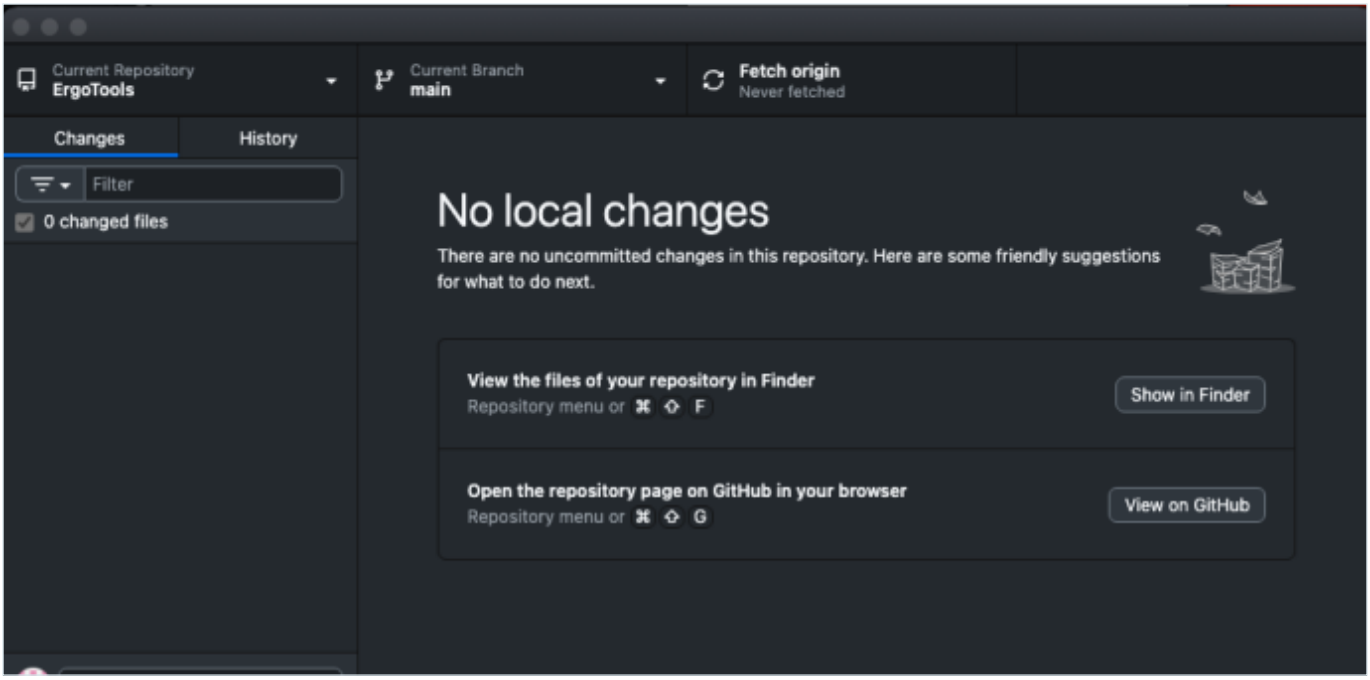
If the button remains Fetch origin after checking, there may be no new commits. Do not clone again just to update. Do not use Commit or Push to receive changes.

Refresh the environment after pulling
Open PowerShell in the checkout. Repeat uv sync and the Windows desktop-library command on page 9, then launch from your existing shortcut.

If Desktop shows local changes, stop and inspect them before pulling. Recovery choices are on page 29.

Update the existing checkout

Save and close ErgoTools. In GitHub Desktop, select ErgoTools and main, then use Fetch origin and Pull origin when offered.



Use the existing repository. Show in Finder helps locate the matching checkout.

Fetching checks for new code. Pulling applies the available commits. If no pull is offered, the checkout can already be up to date.

```
cd "$HOME/ErgoTools-code/ErgoTools"
uv sync --locked --no-dev --python 3.13.14
```

```
uv pip install --only-binary :all: \
  --python .venv/bin/python \
  -r tools/requirements_desktop_trial.txt
```

Launch from the same alias
The alias still points to tools/run_macos.command inside this checkout. It uses the newly updated code and environment.

Refresh code and libraries together

Run these in the repository root after saving and closing the application. Use only the section for your OS.

Linux: pull and synchronize

```
git pull --ff-only origin main
uv sync --locked --no-dev --extra desktop --python 3.13.14
bash tools/run.sh
```

Windows: after pulling in Desktop

```
uv sync --locked --no-dev --python 3.13.14
uv pip install --only-binary :all: `
  --python .venv\Scripts\python.exe `
  -r tools\requirements_desktop_trial.txt
```

macOS: after pulling in Desktop

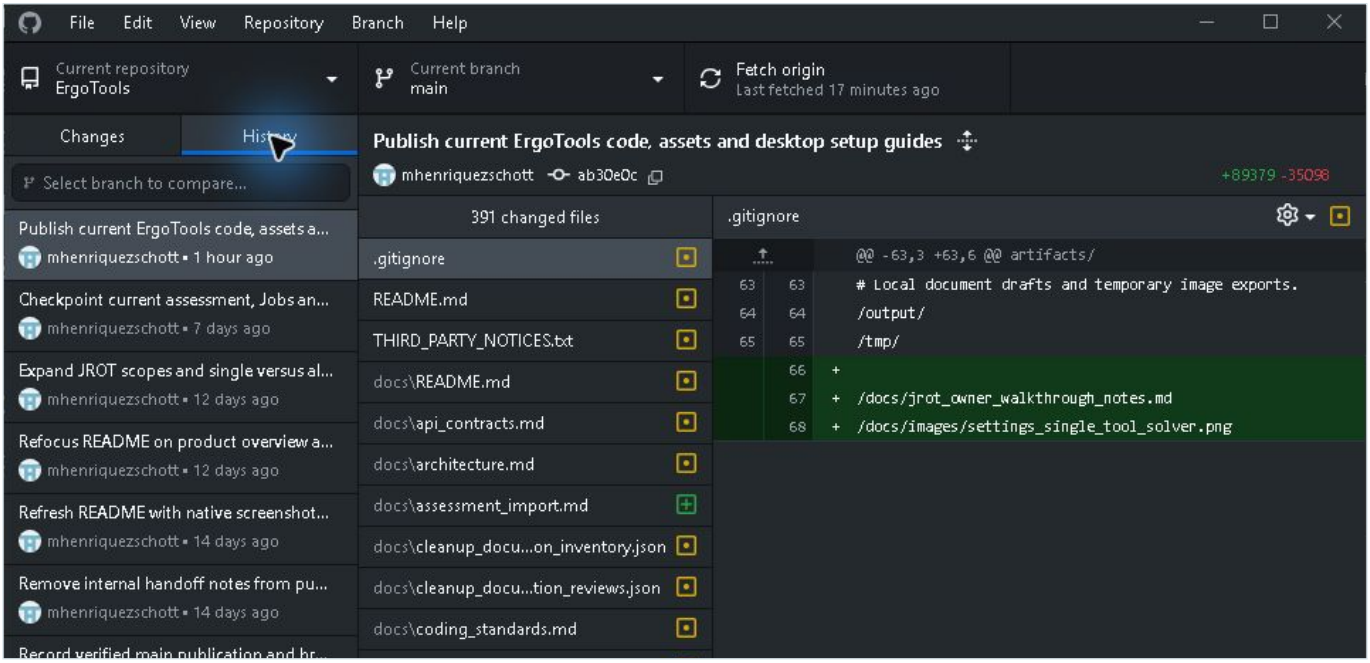
```
uv sync --locked --no-dev --python 3.13.14
uv pip install --only-binary :all: \
  --python .venv/bin/python \
  -r tools/requirements_desktop_trial.txt
```

Why repeat the desktop-library command?
On Windows and macOS, uv sync can remove packages that are outside the core lock selection. Run the desktop requirements command after it, even if those libraries were previously installed.

Linux uses the desktop extra directly. Keep uv.lock and the requirements file from the repository; do not edit pins to resolve a local installation error.

Record the code version before a test

A commit ID identifies the exact source revision. It is more useful than saying “the latest version.”



GitHub Desktop: open History, select the latest commit and use its contextual menu to copy the SHA.

In Windows/macOS Desktop, use **History**, right-click the most recent commit and choose **Copy SHA**. Paste it into your test notes or bug report.

Terminal alternative

```
git rev-parse HEAD
```

Record the conditions too

Operating system/version: _____

Code commit: _____

Display resolution and scale: _____

Light/dark appearance: _____

Project and action tested: _____

The terminal command requires Git on the command path. GitHub Desktop users can use Copy SHA without installing a separate Git command-line tool.

Recover without losing your work

Do not force an update, discard local changes or delete a checkout before understanding what it contains.

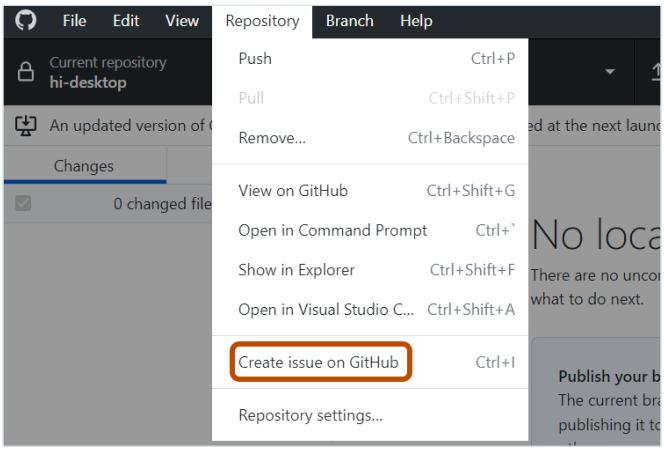
What you see	What to check
Local changes in Desktop	Did you edit code, documentation or a bundled sample? Back up those files and ask for help if unsure. Your personal projects should be outside the checkout.
A branch other than main	Choose the agreed branch. Do not merge branches just to obtain a release.
The shortcut opens old code	Check its target. You may have two checkouts; use Show in Explorer/Finder to locate the intended one.
Python or packages no longer found	Return to the correct checkout and repeat that OS environment setup. .venv is local to that folder.
A moved or copied .venv	Recreate it with the setup commands. Do not share environments between OSs or machines.
An older code version is needed	Ask the team for the exact commit and instructions; back up projects before opening them in older code.

Keep one known working path
Write down the checkout location and last working commit before troubleshooting. A separate fresh clone can help diagnosis, but do not overwrite or delete your existing work.

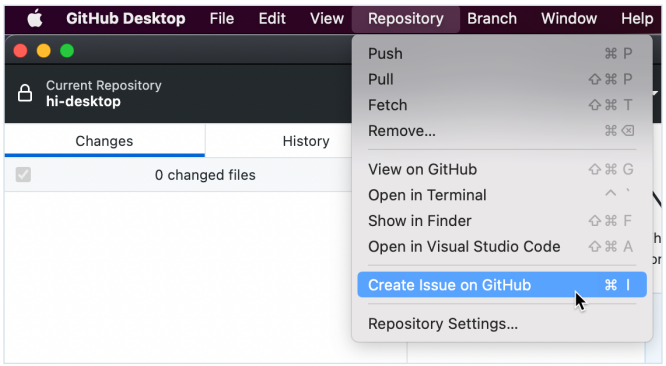
Open the issue page

Use your GitHub account. Search existing issues first; add evidence to the same issue if it already describes your problem.

Open ErgoTools issues in your browser, then choose **New issue**. You can also start from GitHub Desktop:



Windows: Repository > Create Issue on GitHub.



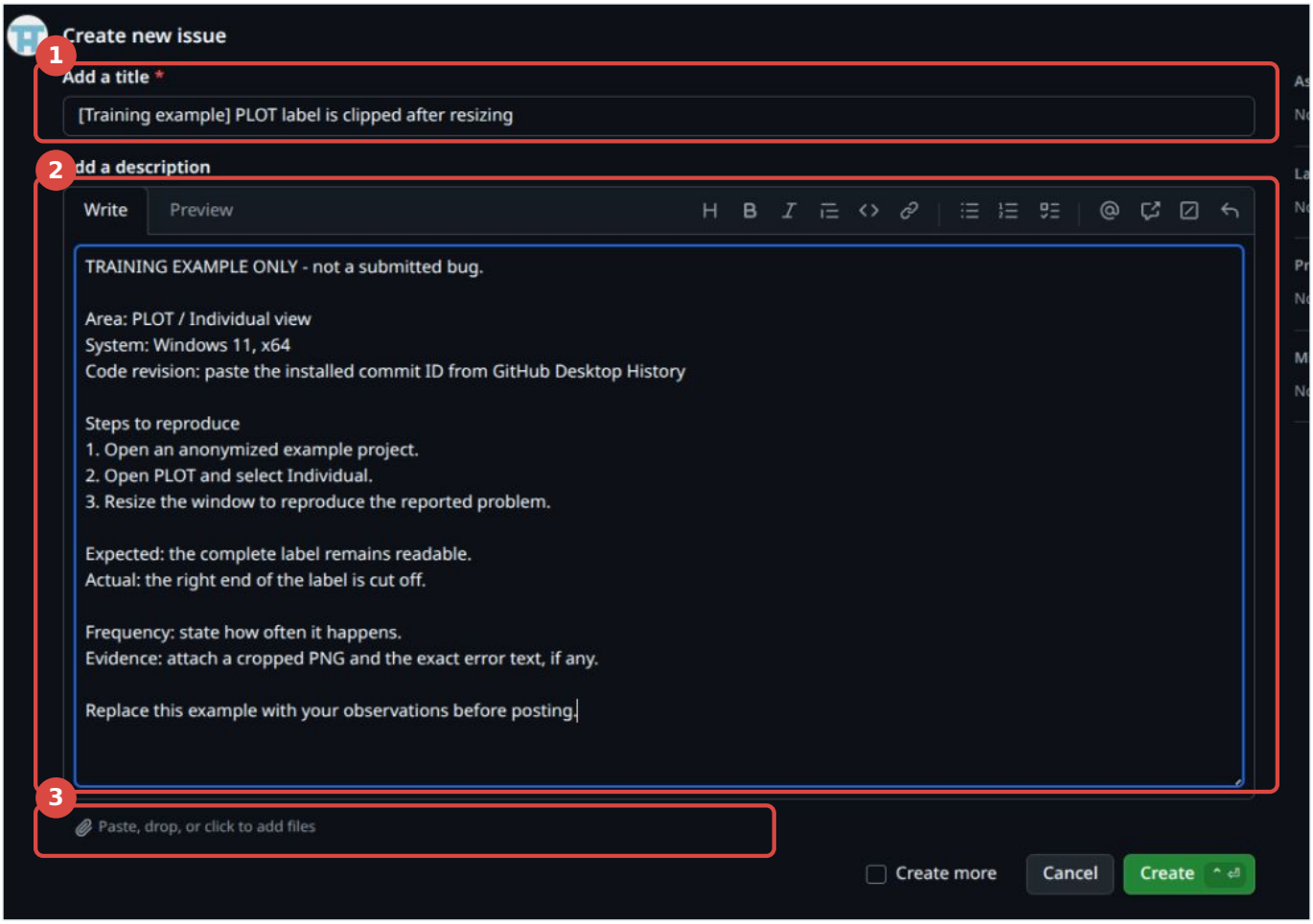
macOS: Repository > Create Issue on GitHub.

Desktop opens the repository's issue page in your browser. The issue form and file attachments are completed there; Desktop is the entry point, not a separate bug tracker.

Use the correct repository
Check the browser address: github.com/mhenriquezschott/ErgoTools. Signing in may be required before New issue is available.

Write a report someone can reproduce

This is an example draft in the ErgoTools repository. Use your own observation, not the example values.



1 Give a precise title. 2 Describe steps and expected/actual behavior. 3 Attach evidence before submitting.

One concrete problem per issue is easier to investigate. Avoid a title such as “does not work”; identify the window and action that fail.

A useful report template

Copy this structure into the description, then replace each bracketed item with your own information.

```
Title: [Window/action and visible problem]

ErgoTools commit: [Copy SHA from Desktop History]
OS and version: [e.g. Windows 11 / macOS Ventura]
Display: [resolution, scale, light/dark appearance]
Project: [new/sample/personal copy; no private data]

Steps to reproduce:
1. [Open this window/project]
2. [Select these settings or enter these values]
3. [Press this button]

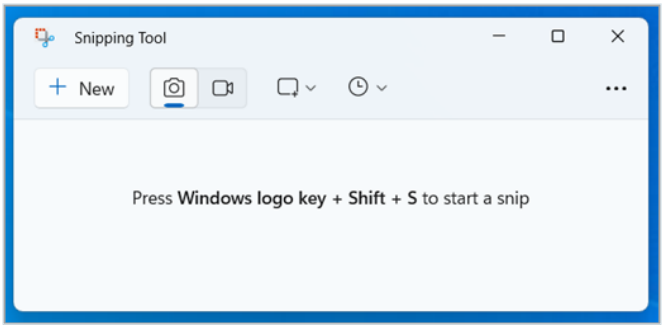
Expected: [what should happen]
Actual: [what happens, exact error if available]
Frequency: [every time / sometimes / once]
Last working version: [if known]
Attachments: [screenshot, diagnostic ZIP, console TXT]
```

Make a small example when possible
If a problem depends on a project, reproduce it in a non-sensitive copy with only the data needed. Share a project only when you have permission to disclose its contents.

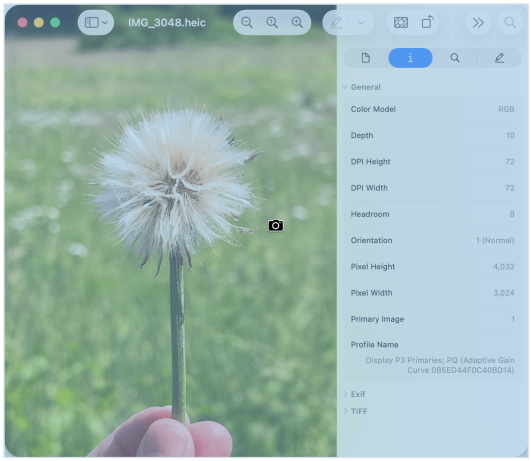
For an optimization issue, include selected tools, block durations, worker restrictions and whether the result was newly calculated or loaded from a saved project.

Capture the part that explains the issue

Keep the relevant window, title and controls visible. Remove unrelated desktops, notifications, email and private project information.



Windows: Snipping Tool can capture a window or rectangle.



macOS: Screenshot offers window and selected-area capture.

Windows

Open Start, search **Snipping Tool**, choose New and select the window or region. Save as PNG in a folder you can find. Windows + Shift + S is an optional shortcut.

macOS

Open Screenshot from Applications > Utilities or Spotlight. Select a window or area; in Options choose a known Save to location such as Desktop. Shift + Command + 5 is optional.

Keep an original and annotate a copy
A rectangle or numbered marker can identify the problem. Do not cover the error text or crop away the control that caused it. Include a second wider image when the surrounding layout matters.

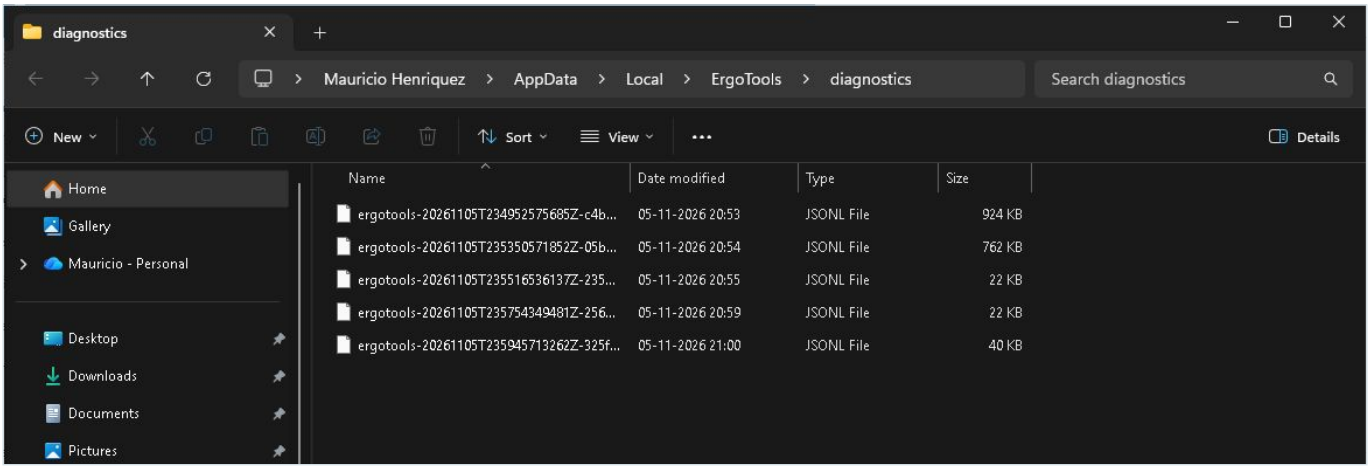
Linux: use your desktop's screenshot application or Print Screen menu. Include display scaling information when reporting clipped or overlapping controls.

Find the launch log

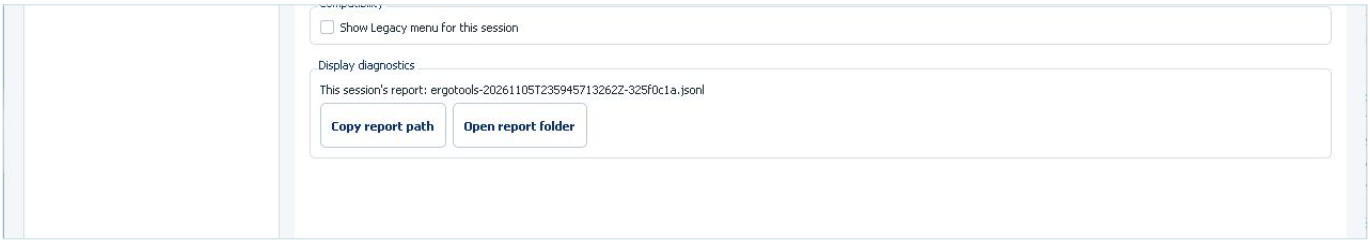
Windows saves startup/display diagnostics automatically. The report folder is separate from your code and project files.

```
%LOCALAPPDATA%\ErgoTools\diagnostics
```

Paste that path into the **Explorer address bar** and press Enter. %LOCALAPPDATA% expands to your own user's local application-data folder; do not replace it with the username from a capture.



Explorer shows the diagnostic files. Use Date modified to find the launch that reproduced the issue.



You can also use Edit > Settings > General > Display diagnostics > Open report folder.

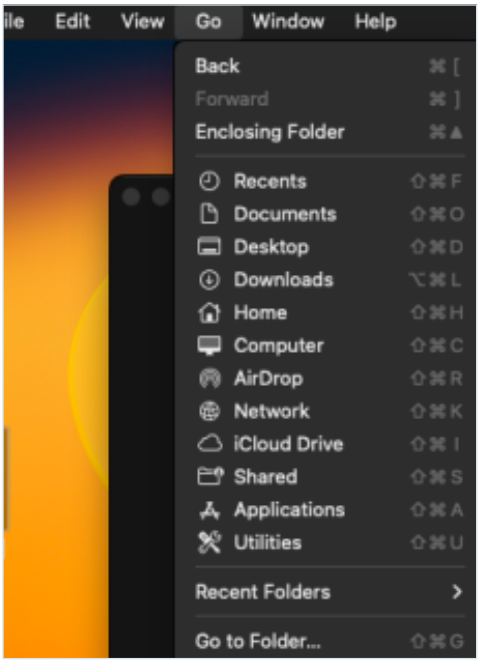
Select the failing launch
Files are named ergotools-…jsonl. The filename uses UTC; Explorer may display local time. Include the matching rotated .jsonl.1 file when present. A later launch creates a different report.

Open the log folder in Finder

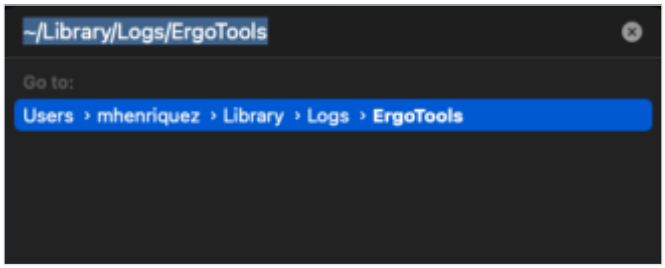
macOS saves startup/display diagnostics automatically. Use Finder rather than navigating through hidden Library folders manually.

```
~/Library/Logs/ErgoTools
```

In Finder choose **Go > Go to Folder**. Paste the path above and confirm. Here ~ means your home folder; keep it exactly as shown.



Choose Go to Folder in Finder.



Paste ~/Library/Logs/ErgoTools and open the result.

Choose the right file
Find ergotools-...jsonl from the launch that showed the problem. Include its .jsonl.1 sibling if present. Record your local test time if the filename timestamp seems different: filenames use UTC.

The same folder is accessible from Edit > Settings > General > Display diagnostics > Open report folder.

Enable a report for a test launch

Linux diagnostics are off by default. Enable them for the launch you want to investigate.

```
bash tools/run.sh --startup-diagnostics
```

Configuration	Folder
XDG_STATE_HOME is set	$XDG_STATE_HOME/ergotools/diagnostics
Otherwise	~/.local/state/ergotools/diagnostics

Use Settings > General > Display diagnostics > Open report folder, or open the relevant path in your file manager. To choose an explicit folder for a test:

```
bash tools/run.sh \
  --diagnostics-dir "$HOME/ErgoTools-diagnostics"
```

What a diagnostic report contains

Reports help identify OS/runtime versions, Qt platform, screen geometry, scale and application events. They are useful for layout failures and some startup problems. They are bounded logs, not a complete operating-system crash dump.

A hard crash may need console evidence too
If the application closes without a useful message in the JSON-lines report, capture a terminal launch as described on the next page.

Launch with console diagnostics

Save your work first. Open a terminal in the correct checkout. These commands add Python fault reporting to the normal launcher.

Windows / PowerShell

```
cd "$HOME\ErgoTools-code\ErgoTools"
$env:PYTHONFAULTHANDLER = "1"
.\tools\run_windows.cmd
```

macOS / Terminal

```
cd "$HOME/ErgoTools-code/ErgoTools"
PYTHONFAULTHANDLER=1 ./tools/run_macos.command
```

Linux / terminal

```
cd "$HOME/ErgoTools-code/ErgoTools"
PYTHONFAULTHANDLER=1 bash tools/run.sh --startup-diagnostics
```

Reproduce the problem once. Keep the terminal open. Select and copy the error text into Notepad or a plain-text editor, then save it as console.txt. Include the command and the final error lines. In macOS TextEdit, choose Format > Make Plain Text before saving console.txt.

Send both kinds of evidence
A console error and the matching launch log complement each other. No console output does not mean that nothing failed; attach the log and exact reproduction steps anyway.

Review, compress and attach

Review the files before selecting them in GitHub. Uploading an attachment starts immediately, even before the issue itself is submitted.

1. Gather a small evidence folder

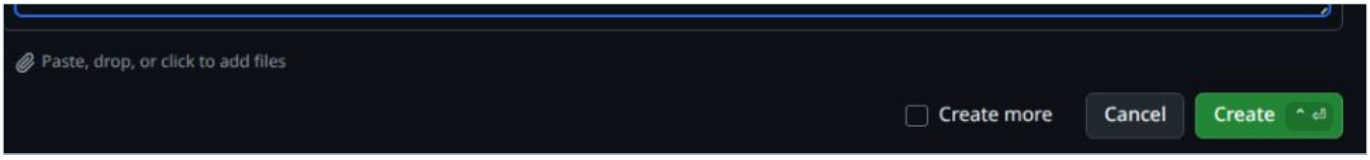
Include the matching .jsonl log and rotated sibling, console.txt if available, relevant PNGs, and a short note with OS, commit and reproduction steps.

2. Review private information

Logs can include local paths, usernames and project identifiers. Screenshots can expose personal data. Redact a copy when necessary and keep the original for private follow-up.

3. Create a ZIP

Windows: right-click the folder and choose Compress to ZIP, or Send to > Compressed (zipped) folder. macOS: right-click > Compress. Linux: use Compress in your file manager.



Click the attachment link below the description, or drag your reviewed ZIP into the text area.

4. Wait, then submit

Wait for the attachment link to appear. Add a short description of the files, re-read the report, then choose Create/Submit issue.

Email alternative

If the team asks for email, send the same reviewed ZIP to the team contact provided to you. Do not post sensitive data publicly merely because GitHub accepts the attachment.

AFTER REPORTING

Follow up and check the fix

Keep discussion and replacement evidence in the same issue so the investigation stays together.

1. Watch for questions

GitHub can notify you about replies. Include exact settings or a smaller example when requested; avoid posting passwords, tokens or private source data.

2. Install the indicated update

Save and close the app, Fetch/Pull main, refresh the environment for your OS, then start from your existing shortcut or alias.

3. Repeat the original steps

Use a personal project copy and the same settings. Report the tested commit and whether the problem is resolved.

Symptom	Useful evidence
Text overlaps or is clipped	Window screenshot, screen resolution/scaling, system appearance, launch log.
Optimization fails or closes the app	Selected tools, restrictions, block times, fresh run steps, launch log and console text.
A project does not open	Exact message, file origin/version, non-sensitive example if permitted.
A launcher stops working	Shortcut/alias target, actual checkout folder, terminal command and error.

Describe what changed

“Works after commit … with the same test project” is more useful than “fixed.” If a different problem appears, open a separate issue and link the related one.

QUICK REFERENCE

Keep this page nearby

Save these locations and your own checkout path before you need to troubleshoot.

Item	Location / action
Repository	github.com/mhenriquezschott/ErgoTools
Bug reports	ErgoTools / Issues
Windows launcher	tools/run_windows.cmd
macOS launcher	tools/run_macos.command
Linux launcher	bash tools/run.sh
Windows logs	%LOCALAPPDATA%/ErgoTools/diagnostics
macOS logs	~/Library/Logs/ErgoTools
Linux logs	~/.local/state/ergotools/diagnostics, unless XDG_STATE_HOME is set
Copyable command companion	docs/guides/installation_commands.md in the repository

Before an update
Save and close ErgoTools → Fetch/Pull main → refresh libraries → relaunch from the existing link → check your project copy.

Before a bug report
Record commit, OS and display settings → reproduce once → capture screenshots and matching logs → review privacy → ZIP and attach.

My checkout: _____
My project folder: _____
Last working commit: _____

Sources and further help

Use these official references for platform-specific details. The repository README remains the concise command reference.

ErgoTools repository and installation instructions

Create a GitHub account

GitHub Desktop downloads

GitHub Desktop supported operating systems

Create an issue from GitHub Desktop

Attach files to GitHub issues

uv installation

uv Python installation

Windows Snipping Tool

Take screenshots on a Mac

Images

Windows and macOS installation and application captures were supplied during ErgoTools installation testing. Crops preserve the real interface; numbered boxes are explanatory overlays. Linux application captures were made from private sample-project copies.

The Repository > Create Issue menu images are from GitHub Docs. The screenshot-tool illustrations are from Microsoft Support and Apple Support, retained with their source records. The issue-form example shows the ErgoTools repository; it is an illustrative draft.

Interface wording may vary slightly with operating-system and GitHub Desktop versions. Follow the named action and your own folder location.